

Hướng Dẫn Làm Labtainer

Giấu tin bằng LSB + header bí mật | lsb-secret-header

Dành cho sinh viên. Tài liệu này hướng dẫn cách đọc yêu cầu, hoàn thiện chương trình, chạy kiểm tra và nộp kết quả. Không chứa lời giải hoàn chỉnh.

1. Mục Tiêu Bài Lab

- Hiểu nguyên lý giấu tin bằng kỹ thuật LSB trong ảnh RGB.
- Tính dung lượng giấu tin tối đa của ảnh RGB.
- Tạo packet có MAGIC, LENGTH, SHA-256 và PAYLOAD.
- Nhúng và trích xuất thông điệp từ ảnh.
- Kiểm tra tính toàn vẹn bằng SHA-256.
- Phát hiện ảnh không có header bí mật.

2. Cài Và Mở Lab

Cài lab bằng RAW URL do giảng viên cung cấp:

```
imodule https://github.com/vuxvinh/KiThuatGiauTin/raw/main/lsb-secret-header.tar
```

Mở lab:

```
labtainer lsb-secret-header
```

Nếu muốn làm lại từ đầu:

```
labtainer -r lsb-secret-header
```

Kiểm tra file trong container:

```
ls
```

- README.md
- cover.png
- no_secret.png
- secret.txt
- encode.py
- decode.py
- check_work.sh
- make_assets.py

3. File Quan Trọng

File	Ý nghĩa
cover.png	Ảnh gốc dùng để giấu thông điệp.
no_secret.png	Ảnh không có thông điệp, dùng cho negative test.
secret.txt	Thông điệp bí mật cần giấu.
encode.py	Chương trình cần hoàn thiện để tạo encoded.png.
decode.py	Chương trình cần hoàn thiện để tạo recovered.txt.
analysis.txt	File sinh viên tự tạo để trả lời bài toán dung lượng.
check_work.sh	Script kiểm tra từng task pass/fail.

4. Kiến Thức Nền

Ảnh RGB gồm nhiều pixel. Mỗi pixel có 3 kênh màu R, G, B. Mỗi kênh màu là một số từ 0 đến 255, biểu diễn bằng 8 bit.

Kỹ thuật LSB, viết tắt của Least Significant Bit, giấu dữ liệu vào bit cuối cùng của mỗi kênh màu.

```
Giá trị ban đầu: 11010110
Mẫu giấu bit 1: 11010111
Mẫu giấu bit 0: 11010110
```

5. Định Dạng Packet

[MAGIC 8 bytes][LENGTH 4 bytes][SHA256 32 bytes][PAYLOAD]

- MAGIC là header bí mật: PTITLSB1.
- LENGTH là độ dài PAYLOAD, lưu bằng 4 bytes big-endian.
- SHA256 là mã băm SHA-256 của PAYLOAD.
- PAYLOAD là nội dung bí mật đọc từ secret.txt.

Nếu decode ảnh không có header, chương trình phải in:

```
NO_SECRET_HEADER
```

6. Các Task Cần Pass

Task	Tên	Điều kiện pass
1	Encode image	Tạo được encoded.png và in ENCODE_OK.
2	Detect secret header	decode.py phát hiện PTITLSB1 và in HEADER_OK.
3	Verify SHA-256	Hash của payload đúng và in SHA256_OK.
4	Recover original message	recovered.txt giống secret.txt.

Task	Tên	Điều kiện pass
5	Detect image without secret	ab_secret.png cho kết quả NO_SECRET_HEADER.
6	Calculate capacity	analysis.txt có đúng 4 dòng dung lượng.

7. Hướng Dẫn Làm encode.py

build_packet(payload)

- Dùng `struct.pack(">I", len(payload))` để tạo LENGTH 4 bytes big-endian.
- Dùng `hashlib.sha256(payload).digest()` để tạo SHA256 32 bytes.
- Trả về MAGIC + LENGTH + SHA256 + payload.

capacity_bits(img)

- Ảnh RGB có width * height pixel.
- Mỗi pixel có 3 kênh màu.
- Mỗi kênh giấu được 1 bit.
- Công thức: width * height * 3.

embed_packet(img, packet)

- Chuyển packet thành dãy bit theo thứ tự bit cao đến bit thấp.
- Duyệt pixel từ trái sang phải, từ trên xuống dưới.
- Với mỗi bit, thay LSB của từng kênh R, G, B.
- Tạo ảnh mới và lưu ra encoded.png.

Công thức thay LSB:

```
new_value = (old_value & 0xFE) | bit
```

8. Hướng Dẫn Làm decode.py

image_lsb_bits(img)

- Chuyển ảnh sang RGB.
- Duyệt từng pixel theo thứ tự `getdata()`.
- Với mỗi pixel, yield r & 1, g & 1, b & 1.

bits_to_bytes(bits)

- Gom mỗi 8 bit thành 1 byte.
- Bit đầu tiên là bit có trọng số cao nhất.
- Tạo byte bằng cách dịch trái và OR với bit hiện tại.

read_bytes_from_lsb(img, nbytes)

- Đọc $nbytes * 8$ bit từ LSB của ảnh.
- Chuyển các bit đã đọc thành bytes.

Quy trình decode

- 1 Đọc 44 bytes header đầu tiên.
- 2 Kiểm tra 8 bytes đầu có bằng PTITLSB1 không.
- 3 Nếu không đúng, in NO_SECRET_HEADER.
- 4 Nếu đúng, đọc LENGTH và SHA256.
- 5 Đọc tiếp PAYLOAD theo LENGTH.
- 6 Tính SHA-256 của PAYLOAD và so sánh.
- 7 Nếu đúng, ghi PAYLOAD ra recovered.txt.

9. Chạy Thử Từng Bước

Chạy encode:

```
python3 encode.py cover.png secret.txt encoded.png
```

Chạy decode:

```
python3 decode.py encoded.png recovered.txt
```

So sánh thông điệp:

```
cmp secret.txt recovered.txt  
echo $?
```

Chạy negative test:

```
python3 decode.py no_secret.png recovered_empty.txt
```

10. Tạo analysis.txt

Với ảnh cover.png kích thước 256x256 RGB:

```
capacity_bits = 256 * 256 * 3 = 196608  
capacity_bytes = 196608 / 8 = 24576  
header_bytes = 8 + 4 + 32 = 44  
max_payload_bytes = 24576 - 44 = 24532
```

Nội dung file analysis.txt cần có:

```
capacity_bits=196608  
capacity_bytes=24576  
header_bytes=44  
max_payload_bytes=24532
```

11. Chạy checkwork

```
bash check_work.sh
```

Kết quả mong muốn:

```
TASK1_ENCODE_IMAGE_PASS
TASK2_HEADER_DETECTED_PASS
TASK3_HASH_VALID_PASS
TASK4_RECOVERED_MATCH_PASS
TASK5_NEGATIVE_TEST_PASS
TASK6_CAPACITY_ANALYSIS_PASS
ALL_TASKS_PASS
```

Có thể chạy checkwork của Labtainer:

```
checkwork lsb-secret-header
```

12. Lỗi Thường Gặp

- Sai thứ tự bit: byte phải được tách từ bit 7 về bit 0.
- Sai endian của LENGTH: phải dùng big-endian với `struct.pack(">I", length)`.
- Không convert ảnh sang RGB: luôn dùng `Image.open(path).convert("RGB")`.
- Ghi sai file output: encode tạo `encoded.png`, decode tạo `recovered.txt`.
- Negative test sai: ảnh không có MAGIC phải in `NO_SECRET_HEADER`.
- Task 6 không pass: `analysis.txt` phải có đúng tên key và đúng giá trị.

13. Câu Hỏi Báo Cáo

- 1 Kỹ thuật LSB là gì?
- 2 Vì sao ảnh sau khi giấu tin nhìn gần giống ảnh gốc?
- 3 Header PTITLSB1 dùng để làm gì?
- 4 Vì sao cần trường LENGTH?
- 5 SHA-256 trong bài lab có tác dụng gì?
- 6 Công thức tính dung lượng giấu tin tối đa của ảnh RGB là gì?
- 7 Nếu ảnh là 256x256 RGB thì giấu được tối đa bao nhiêu byte payload?
- 8 Kỹ thuật LSB có an toàn tuyệt đối không? Vì sao?

14. Dừng Lab Và Nộp Bài

Sau khi làm xong và checkwork đã pass, dừng lab:

```
stoplab lsb-secret-header
```

Labtainer sẽ tạo file kết quả trong:

```
~/labtainer_xfer/lsb-secret-header
```

Nếu giảng viên yêu cầu nộp file .lab, nộp file có dạng:

```
<student_id>.lsb-secret-header.lab
```